Goog-Ad: Complete Account and Deployment Setup Guide

Senior Care Stars | October 14, 2026

Step-by-step accounts, credentials, deployment and verification.

Goog-Ad: Complete Account and Deployment Setup Guide

1. What this guide gets you to
   - Setup order
2. Where each credential belongs
3. GitHub and hosting access
   - GitHub
   - Vercel automation access (only if Codex creates the project)
   - Railway automation access (only if Codex creates the project)
4. Clerk: sign-in and organization setup
5. Google Ads: API and OAuth credentials
   - Developer token and account access
   - OAuth client and refresh token
6. HubSpot and native Google conversion delivery
   - Application read access
   - Native Google Ads Data Manager (the sole uploader)
7. PostgreSQL and administrative provisioning
8. Vercel: deploy the web console
9. Railway: deploy the persistent worker
10. Optional: GA4, BigQuery and Google application credentials
    - GA4
    - BigQuery
11. Optional: model providers and telemetry
    - Claude or Gemini
    - Telemetry and alerts
12. Your handoff checklist
    - Required account information
    - Optional information
    - Decisions before live activation
13. Final acceptance and common blockers

Senior Care Stars | Prepared October 14, 2026

Repository: https://github.com/sx3indy/Goog-Ad Google Ads customer: 284-755-8521 (configuration: 2847558521) HubSpot account: Senior Care Stars, 247696140

1. What this guide gets you to

Follow these steps to prepare identities, credentials, hosting and a real read-only deployment. The local application, source adapters and deployment tooling exist. No hosted projects or real integrations have been verified, and the source has not yet been published to GitHub. Setting up accounts does not complete the remaining specification features or authorize live advertising changes.

Required first: GitHub source publication, Clerk, a verified-TLS PostgreSQL database, Vercel, Railway, Google Ads credentials and HubSpot access. Optional later: GA4, BigQuery, Claude/Gemini and telemetry. Dashboard labels can change; use the linked official help if your screen differs.

Never put secret values in this PDF, GitHub, screenshots or chat. Enter them in provider-managed environment settings or the Codex cloud environment’s secure settings. Public account IDs and project URLs can be shared. Keep a password-manager record of secret owners and rotation dates.

Setup order

1. Arrange publication of the reviewed source to sx3indy/Goog-Ad. Do not import the empty repository into hosting yet.
2. Create Clerk and collect the organization/user IDs.
3. Obtain Google Ads and HubSpot access; verify currency, timezone and CRM fields.
4. Create hosting projects and choose a suitable database.
5. Have the administrative database and tenant provisioning commands run.
6. Configure Vercel and Railway, then deploy in read-only mode.
7. Configure native Google Ads Data Manager and reconcile a small cohort.
8. Run hosted acceptance checks. Add optional integrations afterward.

2. Where each credential belongs

- Codex secure environment settings: temporary deployment automation access, and integration credentials only when needed for validation. A connected hosting account does not automatically make an API token available here. Review/save the environment configuration draft and publish it where prompted; saving a draft alone does not inject credentials into the current runtime.
- Vercel Production environment: web configuration, Clerk keys and application DATABASE_URL. No Ads, HubSpot or model tokens are needed by the web console.
- Railway worker Variables: application DATABASE_URL, Ads/HubSpot credentials, account bindings and optional analytics/model configuration.
- Administrative environment only: ADMIN_DATABASE_URL plus application DATABASE_URL while creating roles, applying migrations and provisioning identity. Remove the admin URL from web and worker runtime environments.
- Password manager: recovery and rotation records. A secret entered into Codex does not automatically appear in Vercel or Railway; configure each intended destination separately.

3. GitHub and hosting access

GitHub

1. Confirm https://github.com/sx3indy/Goog-Ad is the intended repository.
2. Have the current reviewed workspace source committed and pushed there. Confirm package.json, vercel.json, Dockerfile.worker, infra/railway.worker.toml and db/ are visible on the deployment branch.
3. Authorize Vercel and Railway's GitHub integrations to this repository. No personal GitHub token needs to be sent in chat; existing Codex Git proxy authentication should be tried first.

Vercel automation access (only if Codex creates the project)

1. Open https://vercel.com/account/tokens and create a token scoped to the intended account/team with a suitable expiration.
2. Store it securely as VERCEL_TOKEN in Codex environment settings.
3. If using a team, open the team's settings and copy its Team ID into VERCEL_TEAM_ID. Leave this unset for a personal scope.
4. You may instead create the project manually using section 8; then automation credentials are not needed for that manual step.

Railway automation access (only if Codex creates the project)

1. Open https://railway.com and choose the workspace that will own Goog-Ad. Locate its Workspace ID in workspace settings; if unavailable, use Railway's documented API/CLI to retrieve it. A workspace name is not the ID.
2. In account settings, create an account/workspace-capable API token and store it as RAILWAY_API_TOKEN in Codex secure settings. The helper uses an account token, not a project-scoped token for a project that does not exist yet.
3. Store the ID as RAILWAY_WORKSPACE_ID. Do not substitute a project or environment ID.
4. The existing commands are npm run hosting – plan and npm run hosting – create. Creation initializes idle projects and records their IDs; it does not deploy code, connect Git, create a database or copy secrets. Manual creation is also supported.

4. Clerk: sign-in and organization setup

1. Open https://dashboard.clerk.com/ and create an application named Goog-Ad.
2. Enable email sign-in and Organizations. Start with the development instance for testing; keep development/preview users and database separate from production.
3. Create an organization named Senior Care Stars. Invite the initial owner, accept the invitation and select that organization after signing in.
4. Open API Keys. Store the publishable key as NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY and the secret key as CLERK_SECRET_KEY. Only the publishable key is browser-visible. Both belong in Vercel; neither is needed by the worker.
5. Record the organization's org_… ID and the owner's user_… ID. These are used in database provisioning, not as substitutes for API keys.
6. Once the actual Vercel/custom HTTPS domain is known, create/configure the Clerk production instance and complete its required domain/DNS verification. Use production keys for the production deployment.
7. Configure the application's sign-in path as /sign-in and post-sign-in destination as /. Set authorized origins/redirect destinations to your actual domain using the settings available in Clerk. The console already contains sign-in and organization selection pages.
8. Set APP_ORIGIN in Vercel to that exact HTTPS origin, with no path, query or trailing application route. Redeploy after adding/changing the publishable key because it is bundled at build time.
9. Have the initial owner and org mapped using section 7. A Clerk organization alone does not grant console access. Clerk roles do not automatically become database roles.

Success: the mapped owner can sign in, select Senior Care Stars and see the correct account. A user without an application membership is denied access. No Clerk webhook is required by the current initial-owner flow; automatic identity lifecycle synchronization is not implemented.

Official help: https://clerk.com/docs/nextjs/getting-started/quickstart

5. Google Ads: API and OAuth credentials

Developer token and account access

1. Sign in to Google Ads as a user with access to customer 284-755-8521. Check whether it is an advertising account or manager account. If it is a manager, provide the actual child advertising customer ID; the ingestion service rejects manager accounts as zero-spend advertising accounts.
2. Open the Google Ads API Center in the appropriate manager account and request/find the developer token. Its access tier must permit the intended real account. A test-account-only token cannot be treated as production access.
3. Store the token as GOOGLE_ADS_DEVELOPER_TOKEN. If authorization is through an MCC, record that manager's unformatted ID as GOOGLE_ADS_LOGIN_CUSTOMER_ID; do not assume it equals the target customer.
4. Record the target account's actual currency and timezone. Do not assume USD or America/New_York; these must match tenant provisioning and HubSpot financial semantics.

OAuth client and refresh token

1. In https://console.cloud.google.com/, create/select a project for this integration. Enable the Google Ads API and configure Google Auth Platform/OAuth consent (audience, branding and the Ads scope).
2. Create an OAuth client using an appropriate supported flow. For a supervised OAuth Playground flow, use a web client with the exact authorized redirect https://developers.google.com/oauthplayground. For an approved local desktop flow, use a desktop client and its supported redirect flow instead.
3. Store the client ID as GOOGLE_ADS_CLIENT_ID and secret as GOOGLE_ADS_CLIENT_SECRET. Use the same client when obtaining its refresh token.
4. In https://developers.google.com/oauthplayground, enable 'Use your own OAuth credentials' if using that route. Request https://www.googleapis.com/auth/adwords, authorize the Google user with the account access, and exchange the authorization code. Request offline access using the supported OAuth flow; use fresh consent when necessary to obtain a refresh token.
5. Store only the resulting long-lived refresh token as GOOGLE_ADS_REFRESH_TOKEN. Do not use the short-lived access token: the application refreshes it automatically. Do not share Playground screenshots containing tokens.
6. Check OAuth app publication/verification requirements with Google. External apps left in Testing commonly issue refresh tokens that expire after seven days for scopes such as Ads; prepare the appropriate production consent status before relying on unattended operation.
7. Set GOOGLE_ADS_API_VERSION=v25 for the current implementation. Future API retirement requires a code/contract review, not merely changing this value. Set GOOGLE_ADS_CUSTOMER_IDS=2847558521 and GOOGLE_ADS_TENANT_ID to the real provisioned tenant UUID.

8. Keep GOOGLE_ADS_WRITES_ENABLED=false and GOOGLE_ADS_VALIDATE_ONLY=true. Supplying keys alone does not enable mutations; account gates and reviewed live validation remain required.

Success: the worker preflight reads the intended customer; currency/timezone match; a sync returns complete account reporting. Do not certify access from a token’s existence alone.

Official help: https://developers.google.com/google-ads/api/docs/get-started/introduction OAuth: https://developers.google.com/google-ads/api/docs/oauth/overview

6. HubSpot and native Google conversion delivery

Application read access

1. Sign in to HubSpot and confirm the account is Senior Care Stars, 247696140.
2. Find the private-app creation flow in Settings / Integrations, or HubSpot's current developer-app interface for your portal. Create an app named Goog-Ad reporting.
3. Grant read access for contacts, deals, their schema/properties and pipeline metadata. Typical scopes include crm.objects.contacts.read, crm.objects.deals.read, crm.schemas.contacts.read and crm.schemas.deals.read; verify the current scope requirements in your portal, including account-info access. No CRM write scopes are needed by this connector.
4. Store the private-app access token as HUBSPOT_PRIVATE_APP_TOKEN in the worker and secure validation environment. Do not use your HubSpot password.
5. Verify lifecycle value salesqualifiedlead and exact deal-stage labels Move-In Completed and Payment Received. Record the pipeline ID if labels exist in multiple pipelines; set HUBSPOT_PIPELINE_ID in that case.
6. Verify the internal property names referral_payout_amount, referral_payment_date and actual_delivery_cost. These must represent actual collected referral revenue, economic collection date and actual costs. Missing costs or payment dates fail ingestion rather than becoming zero.
7. Verify deal_currency_code, or another explicit currency property, correctly describes the payout/cost amounts. Set HUBSPOT_CURRENCY_PROPERTY to that internal name (default deal_currency_code) and HUBSPOT_CURRENCY to the verified three-letter account currency. The app does not perform currency conversion.
8. Set HUBSPOT_ACCOUNT_ID=247696140, HUBSPOT_CUSTOMER_ID=2847558521 and HUBSPOT_TENANT_ID to the same provisioned tenant UUID as Ads.

Native Google Ads Data Manager (the sole uploader)

1. In Google Ads, open Tools / Data Manager and connect HubSpot if the native connector is available to this account. Authenticate the correct portal and inspect the actual granted permissions.
2. Verify supported objects, matching keys, consent, deduplication and update/refund behavior in the connector. The app's private-app token does not automatically authenticate this separate Google connection.
3. Configure qualified_lead from the sales-qualified lifecycle milestone, converted_lead from completed move-in, and purchase from collected payment. Verify the import/event naming actually supported by your Data Manager UI.
4. Preserve revenue and cost separately. If supported by your HubSpot subscription, create a workflow/calculated property for referral_payout_amount minus actual_delivery_cost. Record its actual internal name and map it to the paid event's conversion value after verifying currency and semantics.
5. Map referral_payment_date as the economic conversion time where supported. Define handling for missing or negative contribution, cancellations/refunds and late corrections; do not silently clamp values to zero.

6. Configure the corresponding imported conversion actions in Google Ads. Keep diagnostic milestones secondary initially. Do not make the paid event primary for bidding until a small cohort’s event counts, match rates and values reconcile against HubSpot.
7. Confirm connector-specific first-party matching and consent requirements. Email/phone/name availability alone does not prove eligible matching. This application does not fetch those PII fields or send a second conversion upload.

Success: portal/stage/property preflight passes, outcome sync succeeds, and native imports reconcile with real paid records. Connector availability and your account’s field support still need live confirmation.

Official help: https://developers.hubspot.com/ and https://support.google.com/google-ads/ (search Data Manager HubSpot).

7. PostgreSQL and administrative provisioning

1. Choose a managed PostgreSQL database in a region near Vercel/Railway. It must be reachable from both hosts and offer a verifiable TLS certificate. Railway's default database template must be checked for this capability; a generated URL alone is not proof. An external managed PostgreSQL provider is acceptable if the template cannot meet it.
2. Obtain an administrative URL and a separate application URL with username ads_app. URLs must target the same dedicated database. Generate a strong application password and URL-encode special characters as required by the provider.
3. Use a public/reachable certificate-matching hostname for Vercel. Use sslmode=verify-full. Do not disable certificate validation. A Railway private-only address is not reachable from Vercel.
4. Worker connections must be direct or session-pooled. Do not use transaction pooling for the worker: its execution and ingestion locks require a retained database session. The web can use a compatible transaction-pooled endpoint.
5. In a trusted administrative environment, securely set APP_MODE=live, ADMIN_DATABASE_URL and DATABASE_URL. Use Node 24 and run the following from the repository root:

```
npm ci
npm run db:provision-role
npm run db:migrate
npm run provision -- /secure/path/tenant.json
```

6. Prepare tenant.json with the template in docs/clerk-setup.md. Replace its placeholders with a newly generated tenant UUID, Clerk org/user IDs and Google-verified currency/timezone. It targets customer 2847558521 and starts with L0 observe-only, allowedActions empty, killed true. Do not reuse demo IDs or seed demo records in production.
7. The role helper verifies that ads_app has no superuser or RLS-bypass capabilities. Migrations currently run through 009_account_reporting. Provisioning establishes the owner mapping and account policy. Additional application memberships need an administrative change; the current CLI provisions the initial owner only.
8. Remove ADMIN_DATABASE_URL from deployed web/worker settings. Configure backups, retention and a real restore rehearsal with your database provider before activation.

Success: application connectivity, restricted role, tenant scope and current migration checks pass. These commands can be completed by Codex once the secure administrative bindings and verified IDs are available; you do not need to paste database passwords into chat.

8. Vercel: deploy the web console

1. After source publication, open https://vercel.com/new and import sx3indy/Goog-Ad. Select the intended team, repository root and Next.js framework. Choose Node 24 and a region near the database where your plan permits.
2. Use the committed vercel.json: install npm ci; build npm run build. Do not configure the persistent worker as a Vercel function.
3. Before a live build, add the following under Project Settings / Environment Variables, selecting Production. Treat DATABASE_URL and CLERK_SECRET_KEY as sensitive. Do not copy production settings into previews.

```
APP_MODE=live
APP_ORIGIN=https://YOUR-ACTUAL-STABLE-DOMAIN
DATABASE_URL=YOUR-ADS_APP-URL-WITH-VERIFIED-TLS
DB_POOL_MAX=2
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=YOUR-PRODUCTION-PUBLISHABLE-KEY
CLERK_SECRET_KEY=YOUR-PRODUCTION-SECRET-KEY
```

4. Record the actual assigned stable project domain or connect a domain you own. Finish Clerk production domain/DNS configuration and make APP_ORIGIN match. Reserve/configure the project before building if necessary to resolve this dependency; do not invent an available domain.
5. Keep deployment protection enabled during acceptance. Deploy/redeploy the source after setting the correct Clerk key. The health path is /api/health; authentication readiness and database migrations must pass.
6. In a trusted environment with equivalent web settings and no admin URL, run npm run deploy:check – web. Then test actual Clerk sign-in, organization selection, console account identity, sign-out and denied memberships in the hosted UI.
7. Record the project URL and final APP_ORIGIN for handoff. Never put the Ads/HubSpot/model tokens in Vercel just to make worker jobs run.

Official help: https://vercel.com/docs/frameworks/full-stack/nextjs and https://vercel.com/docs/environment-variables

9. Railway: deploy the persistent worker

1. Open https://railway.com and create/select the Goog-Ad project in the intended workspace. Add a service connected to the published sx3indy/Goog-Ad repository.
2. Set the service config-file path to infra/railway.worker.toml and repository root to the root. That configuration selects Dockerfile.worker and npm run worker:live. Do not select the alternative web template.
3. Do not generate a public domain for this worker. It runs a continuous background loop, not an HTTP server. Initially use one replica; test recovery before scaling.
4. In worker Variables add the required values below. Enter the real secrets using Railway's managed controls; angle-bracket descriptions below are placeholders, not literal values.

```
APP_MODE=live
DATABASE_URL=<restricted ads_app URL, verified TLS>
DB_CONNECTION_MODE=direct
DB_POOL_MAX=8
GOOGLE_ADS_API_VERSION=v25
GOOGLE_ADS_CUSTOMER_IDS=2847558521
GOOGLE_ADS_TENANT_ID=<new provisioned tenant UUID>
GOOGLE_ADS_DEVELOPER_TOKEN=<developer token>
GOOGLE_ADS_CLIENT_ID=<OAuth client ID>
GOOGLE_ADS_CLIENT_SECRET=<OAuth client secret>
GOOGLE_ADS_REFRESH_TOKEN=<refresh token>
GOOGLE_ADS_WRITES_ENABLED=false
GOOGLE_ADS_VALIDATE_ONLY=true
HUBSPOT_ACCOUNT_ID=247696140
HUBSPOT_CUSTOMER_ID=2847558521
HUBSPOT_TENANT_ID=<same tenant UUID>
HUBSPOT_CURRENCY=<verified three-letter currency>
HUBSPOT_PRIVATE_APP_TOKEN=<private-app access token>
HUBSPOT_CURRENCY_PROPERTY=deal_currency_code
```

5. Add GOOGLE_ADS_LOGIN_CUSTOMER_ID only when MCC authorization requires it; add HUBSPOT_PIPELINE_ID if stage labels are ambiguous. Use DB_CONNECTION_MODE=session only for a verified session pool. Override the currency property if a different verified property is needed.
6. Do not add ADMIN_DATABASE_URL, Clerk keys or hosting automation tokens to worker runtime. Deploy after database provisioning completes.
7. Run npm run deploy:check – worker in a trusted environment with equivalent variables and database access. This performs read-only Ads access and HubSpot metadata checks. Inspect Railway logs for worker.started and source-job status; a running process alone is insufficient.
8. Verify completed Google Ads and HubSpot jobs in the console, actual account spend and business financial data. Restart the worker and confirm jobs/recovery continue without duplicate mutations or lost evidence.
9. Record the Railway project/service URLs for handoff. The service does not need a public worker URL.

Official help: https://docs.railway.com/ (deploying from GitHub, config as code, variables and API authentication).

10. Optional: GA4, BigQuery and Google application credentials

These do not block the initial Ads/HubSpot read-only foundation. They do require actual access before their jobs can be certified. The current Codex environment already supplies a Google Application Default Credentials binding; reuse it if authorized. That binding is not automatically available in Railway.

GA4

1. Find the numeric GA4 Property ID under Admin / Property details; it is not the G-… measurement ID. Verify the property's Google Ads link to the target account and event/key-event definitions.
2. Enable Google Analytics Data API in the Google Cloud project. Give the chosen service-account/principal Viewer access to that GA4 property.
3. On the worker set GA4_PROPERTY_ID, GA4_TENANT_ID (same tenant UUID) and GA4_CUSTOMER_ID=2847558521.
4. Provide supported Google ADC authentication. Prefer an available workload identity mechanism. If a service-account JSON file is required on this host, deliver it as a secure file and set GOOGLE_APPLICATION_CREDENTIALS to its runtime file path. The variable is a path, not raw JSON. The repository does not yet contain an automated Railway secret-file installer; arrange secure delivery before enabling these jobs.
5. Verify GA4 jobs and account filtering. Thresholded/truncated reports are rejected; GA4 key events are not automatically proof of collected payment.

BigQuery

1. Enable BigQuery API and select a billed project and dataset region consistent with your governance requirements.
2. Create the dataset and audit_facts table using infra/bigquery.sql, replacing its project/dataset placeholders. Review its schema and deduplicating query before applying it.
3. Grant the runtime principal the minimum dataset-level insert permissions (for example, suitable BigQuery Data Editor permissions on only that dataset); grant broader setup/query permissions only to the operator that needs them.
4. Set BIGQUERY_PROJECT_ID, BIGQUERY_DATASET and BIGQUERY_TENANT_ID on the worker. Reuse its authorized ADC principal if appropriate.
5. Verify insert permissions, export jobs, row errors and deduplicated audit queries. Export is at-least-once; do not assume raw row counts are exactly-once.

Official help: https://developers.google.com/analytics/devguides/reporting/data/v1 and https://cloud.google.com/docs/authentication/application-default-credentials

11. Optional: model providers and telemetry

Claude or Gemini

1. Choose providers based on budget and approved data-handling rules. Neither is needed for initial reporting, and neither gets direct mutation authority.
2. For Claude, create a key in https://console.anthropic.com/ and set ANTHROPIC_API_KEY plus ANTHROPIC_MODEL on the worker. Select an exact model identifier currently available to your account; do not copy an unverified example.
3. For Gemini, create a key in https://aistudio.google.com/ and set GEMINI_API_KEY plus GEMINI_MODEL. Check Google Cloud billing/access and suitable key restrictions for the Generative Language API.
4. Configure billing limits/provider controls. Verify an explicit model-sync job produces schema-valid proposals and passes evaluation before relying on it. Model jobs are manually requested, not part of the automatic source schedule.

Telemetry and alerts

1. Choose an OTLP/HTTP collector supporting trace delivery. Set OTEL_EXPORTER_OTLP_ENDPOINT to its HTTPS base endpoint in the intended web/worker environment; the implementation appends v1/traces.
2. Configure and verify authentication using supported exporter settings (such as OTEL_EXPORTER_OTLP_HEADERS when appropriate for the installed exporter). Keep header values secret. Confirm actual delivery, redaction and dashboards; a configured URL is not validation.
3. Application failures persist alerts in the console. External paging/email/Slack routing is not implemented merely by providing a token. Choose a destination and complete that integration/acceptance separately if needed for production operations.

12. Your handoff checklist

Enter secrets securely, then send only nonsecret IDs/URLs and confirmation of completion. You can complete this in stages.

Required account information

- GitHub source publication confirmed and chosen deployment branch.
- Vercel project URL, team scope if applicable, final HTTPS domain/APP_ORIGIN.
- Railway workspace ID and project/service URLs.
- Clerk organization ID and initial owner’s user ID; production keys entered securely.
- Database provider/region and pooling mode; admin/app URLs supplied securely for their separate jobs.
- Google Ads advertising customer ID confirmed, MCC login ID if needed, actual currency and timezone; four credential values entered securely (developer token, OAuth client ID/secret, refresh token).
- HubSpot portal 247696140 confirmed; private-app token entered securely; pipeline ID if needed and verified financial/currency property names.
- Native Data Manager connection and conversion-action IDs/settings, with cohort reconciliation results before primary bidding activation.

Optional information

- GA4 numeric property ID and granted principal; BigQuery project/dataset and ADC delivery method.
- Selected Claude/Gemini model IDs, secure keys and approved budgets.
- OTLP collector/notification destination, access, retention and tested delivery.

Decisions before live activation

- Real daily planned budget and protected terms; who approves changes and how application memberships are provisioned.
- Refund/cancellation/negative-contribution rules, attribution windows and consent/retention policies.
- Database backup/restore evidence and incident contact.
- Actual account release-validation evidence and reviewed settings. Keep writes disabled and the initial account killed until this is completed. Credentials and a successful deployment alone do not authorize autonomous operations.

13. Final acceptance and common blockers

1. Hosted /api/health succeeds with current migrations and the restricted app login.
2. Actual Clerk owner login and organization selection succeed; unauthorized memberships are rejected.
3. Ads and HubSpot sync jobs complete, correct account IDs/currency/timezone appear, and missing financial data remains unknown rather than fabricated zero.
4. Native Data Manager conversions reconcile with a small reviewed cohort; no duplicate uploader is introduced.
5. Worker restart preserves durable jobs, audit and recovery. Backups are restored in a disposable environment; telemetry and external notifications are tested if configured.
6. Record what was tested versus still pending. Current local evidence is 39 unit tests, 30 real-database tests, five browser tests, a web production build and worker image checks. The live UI browser test uses a fixture and does not certify Clerk login.

‘Organization is not provisioned’: verify the Clerk org-to-tenant mapping and owner’s database membership. ‘No tenant membership’: the Clerk user ID differs or an application membership is absent. Blank/failed Clerk UI: verify instance/domain/key pairing and rebuild after changing publishable keys.

Database TLS or role failure: use a reachable certificate-matching endpoint and ads_app login; do not disable verification. Worker pooling error: select direct/session, not transaction pooling. HubSpot metadata failure: check portal, scopes, exact internal properties and duplicate stage labels. Ads authorization failure: check developer-token tier, OAuth user/client/refresh pairing and MCC login header. Refresh token expires after a week: review OAuth Testing status and production consent requirements.

Before changing live-write flags, follow docs/release-gates.md. Remaining full-specification subsystems are listed in docs/implementation-status.md; account setup does not certify those features as implemented.